Your Code

```
def preprocess_text(text):
    normalized = text.lower().strip()
    for char in ".,!?;:":
        normalized = normalized.replace(char, "")
    return normalized

def analyze_sentiment(text):
    positive_keywords = ["good", "excellent", "great", "amazing", "happ
    negative_keywords = ["bad", "poor", "slow", "terrible", "worst", "c

    words = text.split()
    pos_count = 0
    neg_count = 0

    for word in words:
        if word in positive_keywords:
            pos_count += 1
        elif word in negative_keywords:
            neg_count += 1

    if pos_count > neg_count:
```

Input

stdin input

Output

Your Code

```
import re

def analyze_password(pw: str):
    if pw is None:
        pw = ""
    length = len(pw)

    has_upper = any('A' <= c <= 'Z' for c in pw)
    has_lower = any('a' <= c <= 'z' for c in pw)
    has_digit = any('0' <= c <= '9' for c in pw)
    has_symbol = any(not c.isalnum() for c in pw)

    # Pattern checks
    is_empty = length == 0
    is_long = length >= 12

    only_one_type = sum([has_upper, has_lower, has_digit, has_symbol])
    all_same_char = length > 0 and len(set(pw)) == 1

    # Repeated substring (simple check)
    repeated = False
    for k in range(1, length // 2 + 1):
        sub = pw[:k]
```

Your Code

```
import sys

def extract_error_lines(log_text):
    error_lines = []
    for line in log_text.splitlines():
        if "ERROR" in line or line.startswith("ERROR") or line.find("e
            error_lines.append(line.strip())
    return error_lines

def classify_error_type(line):
    s = line.upper()
    if "TIMEOUT" in s:
        return "TIMEOUT"
    if "DATABASE" in s:
        return "DATABASE"
    if "AUTH" in s or "LOGIN" in s:
        return "AUTH"
    if "NOT FOUND" in s or "404" in s:
        return "NOT_FOUND"
    if "DISK" in s or "SPACE" in s:
        return "DISK"
    return "OTHER"
```

Input

stdin input

Output

Your Code

```python
def extract_domain(email):
    if "@" in email and email.count("@") == 1:
        parts = email.split("@")
        if parts[0] and parts[1]:
            return parts[1]
    return None

def analyze_email_domains(email_list):
    domain_counts = {}

    for email in email_list:
        domain = extract_domain(email.strip())
        if domain:
            if domain in domain_counts:
                domain_counts[domain] += 1
            else:
                domain_counts[domain] = 1

    if not domain_counts:
        return "No valid domains found."
```

Input

```
stdin input
```

Output

```
Most Common Domain: gmail.com
Frequency: 3
```